

Q1.



```
PS C:\Users\xiayi\Desktop\COMP-409> java a3/q1.java 40 1000
Time taken for q1a: 57 ms
Time taken for q1b: 22 ms
PS C:\Users\xiayi\Desktop\COMP-409> java a3/q1.java 40 1000
Time taken for q1a: 65 ms
Time taken for q1b: 22 ms
PS C:\Users\xiayi\Desktop\COMP-409> java a3/q1.java 40 1000
Time taken for q1a: 61 ms
Time taken for q1b: 22 ms
PS C:\Users\xiayi\Desktop\COMP-409> java a3/q1.java 40 1000
Time taken for q1a: 67 ms
Time taken for q1b: 20 ms
PS C:\Users\xiayi\Desktop\COMP-409> java a3/q1.java 40 1000
Time taken for q1a: 60 ms
Time taken for q1b: 20 ms
```

The above screenshot are example executions of the driver program q1.java, with parameters $k = 40$ and $m = 1000$. Due to the amount of blocking needed to ensure atomic read/write operations, along with blocking the operations, the time taken to perform many operations of q1a.java would be much higher than q1b.java that utilizes atomic packages to allow atomicity and concurrency.

Q1a:

q1a.java includes `get(int i)`, `set(int i, Object o)`, and `resize()` functions and a constructor function to initialize the arrays. This implementation includes a lock for resizing the resizable array, and an array of locks of size $\sqrt{n}$ that correlate with a set of indexes of the array. These locks are used for read/write operations to ensure atomicity on each index of the array by locking the lock that correlates with the index being operated on. $\sqrt{n}$ locks were chosen to ensure that q1a.java only uses $O(n)$ number of locks to control synchronization. This allows threads to concurrently read and write to a set of indexes of the array without attempting to read/write to an index in the same set at the same time that would lead to race conditions.

The resize lock is used to ensure that only a single thread is able to resize the array to ensure that the array is not resized twice at one index call (i.e. if two threads both initially call $i = 20$, then only one of these threads will be able to resize the array). Additionally, it is used to ensure that no thread is allowed to read/write to the array when it is resizing (with the while loop checking if resize is locked).

Q1b:

q1b.java includes `get(int i)`, `set(int i, Object o)`, and `resize()` functions and a constructor function to initialize the arrays. This implementation is lock-free and utilizes an `AtomicReference<Object[]>` resizable array to ensure atomic read/write operations in the array, and an `AtomicBoolean` variable `resizing` to stop other threads from acting on the array while the

array is being resized, and to ensure that only a single thread will resize the array at one index call. The resize function would utilize `compareAndSet` to check the value of the resizing variable and if it is currently false, then it sets resizing to true and continues to resize the array. If it is already set to true, then a different thread is already resizing the array, so the thread would spin until the array is done resizing, ensuring that the array is resized only once at the index. The resizing `AtomicBoolean` variable is also used in get and set, where it is used to check if the array is being resized, and would keep spinning until resizing is set back to false. As the array was initialized as an `AtomicReference<Object[]>` array, the read/write operations conducted are atomic and the array can be concurrently operated.

Q2: The execution scenario tested was conducted by running **java q2.java 100000 t 5**, where t = number of threads. The results of the plot file were extracted by averaging 10 data points for each of the execution scenarios, and calculating the speedup with the formula: $\text{Speedup} = (\text{Average Time for a Single-Thread}) / (\text{Average Time for Multi-Threading})$. This resulted in an average execution time shown in the following table:

Number of threads	Average Execution Time (ms)	Speedup
1	7	1
2	5.8	1.20689655
3	5.3	1.32075472
4	5.2	1.34615385
5	4.9	1.42857143
10	4.5	1.55555556

With the data provided, it is clear that the program generates speedup when the number of threads is greater than 1. This can be seen in the program as each thread concurrently calculates the threads' own individual b, f, and m values without being blocked or interrupted. The only section that blocks threads from concurrently acting is the synchronized critical section from line 79 to line 83 that is used to calculate the final b, f, and m values. Due to this, the time needed to calculate the final b, f, and m values decreases as multiple threads can concurrently decipher the bracket matching into smaller sections, splitting up the work that each thread needs to do.